Predictive Maintenance AI

Actividad 5 - Gestión de proyectos de inteligencia artificial

Módulo 2 / Semana 6 | Entrenamiento, búsqueda de hiperparámetros y trazabilidad con MLflow

Universidad Tecmilenio | Módulo 2 / Semana 6 | Responsable: @isaaccg88

Entrega académica: enlace a repositorio público de GitHub con PDF, código, datos y evidencias MLflow.

Experimento ejecutado en Windows 11 | Python 3.13.5 | scikit-learn 1.9.1 | MLflow 3.17.1

Fecha del experimento: 18 de octubre de 2026 | Fuente: UCI AI4I 2020, DOI 10.24432/C5HS5C.

Modelo seleccionado antes del test: **RandomForest**; umbral validado: **0.20**.

Documento generado a partir de resultados y evidencias reales. Las limitaciones del conjunto sintético se describen explícitamente.

1. Resumen ejecutivo y problema

La manufactura requiere identificar condiciones operativas relacionadas con fallas sin sobreinterpretar un dataset sintético como predicción futura. Esta actividad compara regresión logística y random forest entrenados sobre la misma partición, con GridSearchCV y cinco folds estratificados. MLflow guarda parámetros, métricas y modelos para reproducibilidad.

RESULTADOS PRINCIPALES: Random Forest fue seleccionado exclusivamente en validación (F2 0.7143, umbral 0.20). En test logró F1 0.6341, recall 0.7647 y Average Precision 0.7518. La regresión logística obtuvo F1 0.4096, recall 0.3333 y Average Precision 0.4322. La mejora operativa se acompaña de más falsas alarmas y mayor costo de entrenamiento, discutidos en las secciones 6 y 8.

Reproducibilidad: código Python y notebook publicados en GitHub, semilla 42, splits congelados, SHA256 y entorno Windows 11 con Python 3.13.5, scikit-learn 1.9.1 y MLflow 3.17.1.

2. Procedencia y diccionario del dataset

Fuente original: UCI AI4I 2020 Predictive Maintenance Dataset (DOI 10.24432/C5HS5C, licencia CC BY 4.0). Se parte del snapshot académico de siete columnas usado previamente en la Actividad 4, por lo que no se afirma que sea el archivo bruto completo de UCI.

Variable	*Tipo y significado*
Type	Categoría de tipo de producto: L, M o H.
Air temperature	Temperatura ambiente (K).
Process temperature	Temperatura del proceso (K).
Rotational speed	Velocidad de rotación (rpm).
Torque	Par o torque de operación (Nm).
Tool wear	Desgaste de herramienta (min).
Machine failure	Variable objetivo: 1=falla, 0=sin falla.

3. Gestión, limpieza y versionamiento

PROCEDENCIA VERIFICADA: además del snapshot de siete columnas usado en el entrenamiento, se versiona el archivo oficial UCI de 10,000 filas y 14 columnas. La comparación fila por fila de las siete variables utilizadas produjo cero discrepancias. Se excluyeron UDI y Product ID por ser identificadores y TWF, HDF, PWF, OSF y RNF por representar modos de falla directamente vinculados al objetivo; incluirlos supondría fuga de información de la etiqueta.

SHA256 archivo original de UCI: dc6630cd9b1f0f853922fad78a1b6436570d3f1ec863f1dd5c4340ac56bc8a8e.

Elemento	*Evidencia de trazabilidad*
Fuente primaria	UCI AI4I 2020, DOI 10.24432/C5HS5C, CC BY 4.0
Archivo completo	datos/datos_ini/uci_ai4i2020_original.csv (14 columnas)

Elemento	*Evidencia de trazabilidad*
Snapshot del entrenamiento	datos/datos_ini/ai4i2020.csv (7 columnas)
Comparación	10,000 filas y siete columnas: 0 discrepancias
Registro verificable	docs/uci_source_manifest.json; fuentes/verificar_fuente_uci.py

Snapshot: 10,000 registros; clases {'0': 9661, '1': 339}. Duplicados exactos: 0. Nulos totales: 0. Etiquetas de split: entrenamiento 7,000, validación 1,500, prueba 1,500. Si hubiera duplicados, el pipeline los detectaría y detendría la ejecución para revisar las particiones, en vez de introducir contaminación. La imputación, codificación y escala se ajustan dentro del pipeline de CV.

SHA256 snapshot inicial: 7d266d84d3478c11de16c1764f4d0edd77f85880deef3899b934e7622c007a8a.

SHA256 snapshot limpio: 36099b87dff481812b9c034f575c1e831c3b4f6a36d54c4d44c80979066880f2.

Los valores extremos se identificaron mediante el método IQR; no se eliminaron observaciones extremas válidas sin criterio industrial documentado.

Variable	*Posibles valores extremos (IQR)*
Air temperature	0
Process temperature	0
Rotational speed	418
Torque	69
Tool wear	0

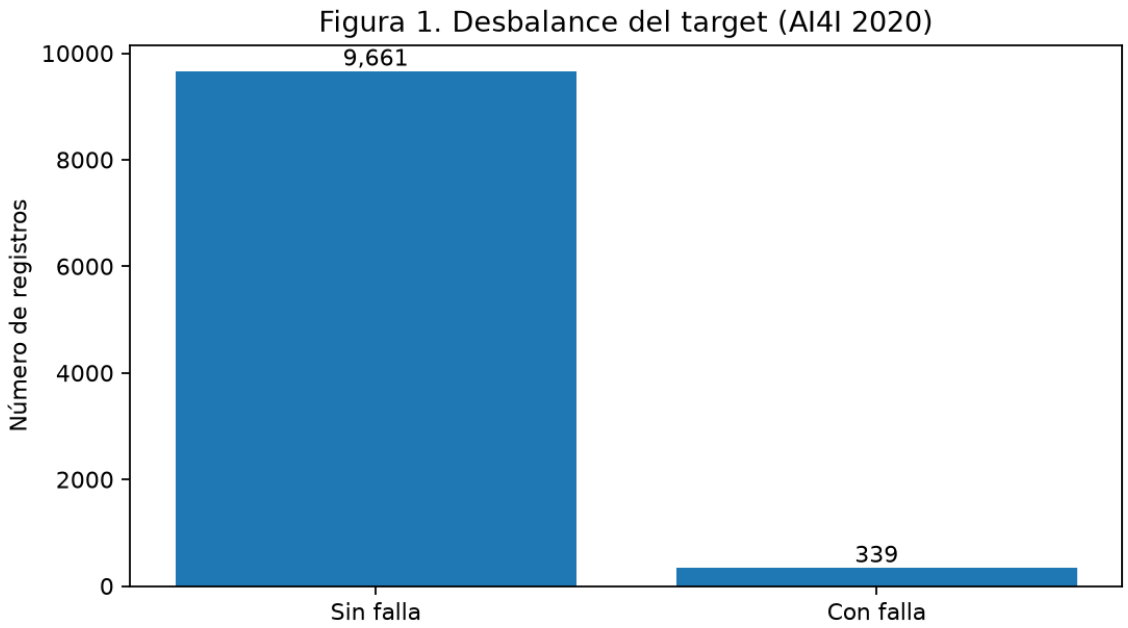


Figura 1. Distribución de clases: 9,661 registros sin falla y 339 con falla; un baseline mayoritario alcanza 96.61 por ciento de accuracy sin detectar fallas.

INTERPRETACIÓN DE LA FIGURA 1: la prevalencia de fallas de 3.39 por ciento exige Average Precision, recall, F1/F2 y revisión de matrices de confusión; accuracy por sí sola no es adecuada.

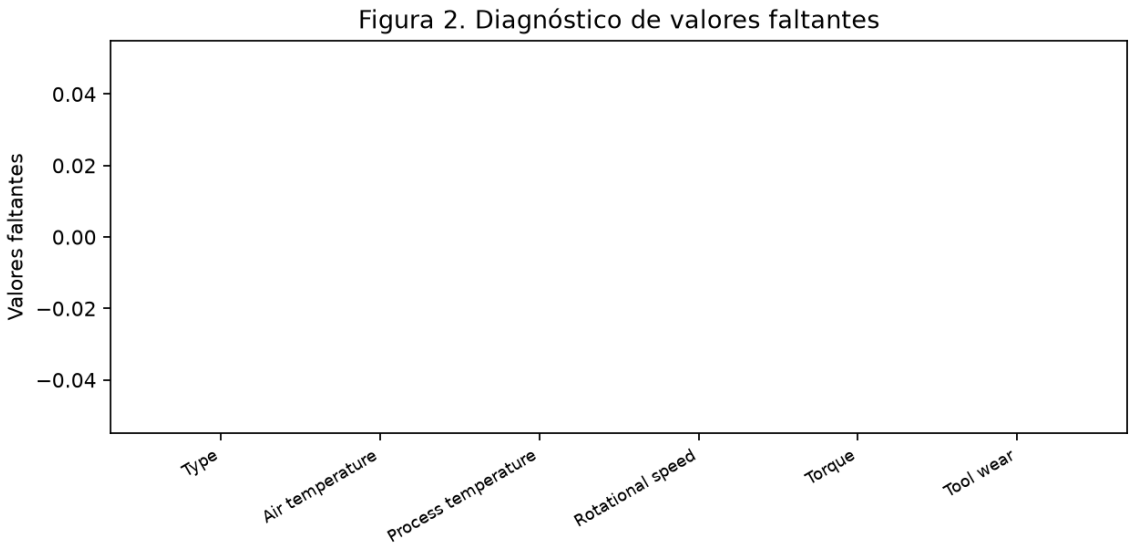


Figura 2. Diagnóstico de valores faltantes: cero nulos en las seis características.

INTERPRETACIÓN DE LA FIGURA 2: en este snapshot no se requirió imputar datos; aun así, el pipeline mantiene SimpleImputer dentro de cada fold para conservar un proceso seguro ante futuras entradas incompletas.

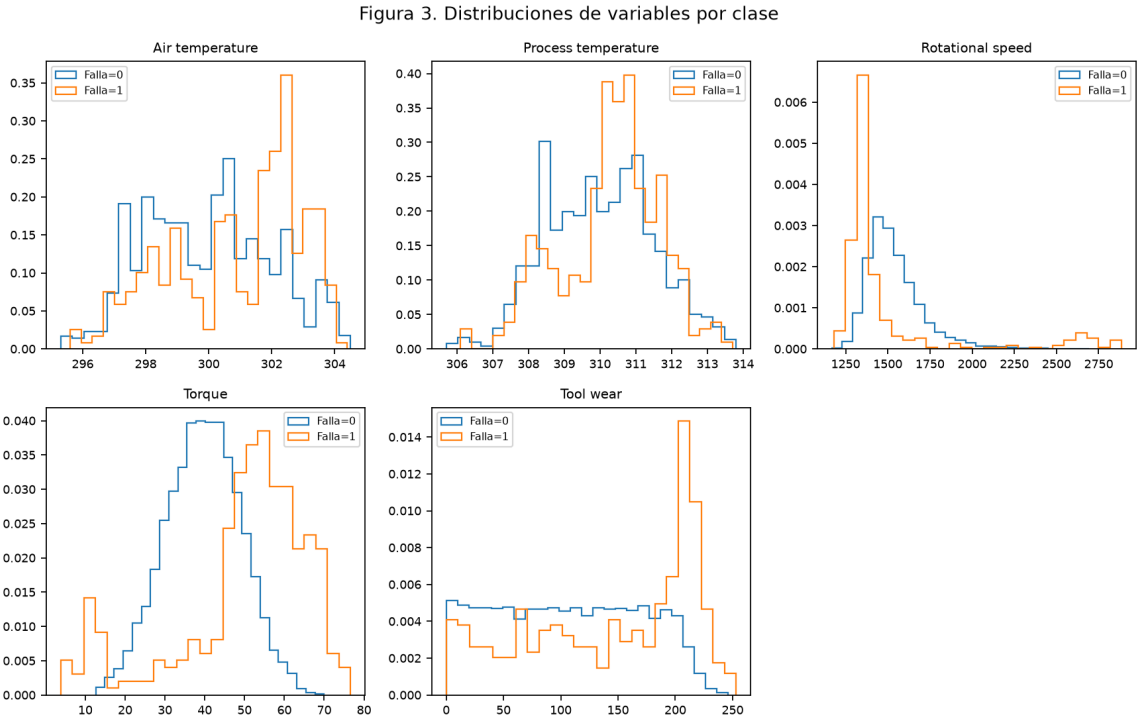


Figura 3. Histogramas de variables numéricas según etiqueta de falla: diferencias descriptivas, no causales.

INTERPRETACIÓN DE LA FIGURA 3: se comparan distribuciones normalizadas por clase, evitando que los 9,661 negativos oculten los 339 positivos. Estas diferencias no implican anticipación temporal ni generalización entre máquinas.

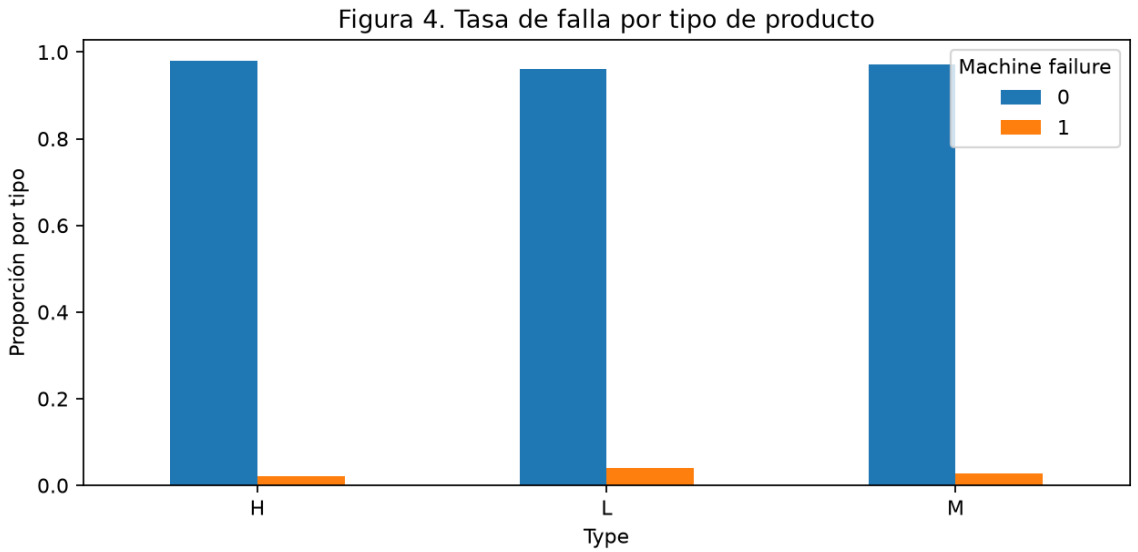


Figura 4. Proporción de falla por categoría Type (L/M/H): diagnóstico descriptivo.

INTERPRETACIÓN DE LA FIGURA 4: se comparan tasas por tipo de producto, no volúmenes absolutos. La variable es categórica y se codifica con OneHotEncoder ajustado dentro de cada fold, no mediante una conversión ordinal arbitraria.

ANÁLISIS DE CALIDAD: 339 de 10,000 registros son positivos (3.39%). La clase mayoritaria alcanza 96.61%, por lo que accuracy aislada es engañosa. No existen valores nulos ni duplicados exactos. El método IQR señala 418 posibles valores extremos en velocidad y 69 en torque. Se conservaron: un valor extremo operativo puede ser precisamente una señal de falla. Las figuras 1 a 4 documentan desbalance, completitud y patrones; las variables numéricas fueron tratadas dentro del Pipeline para evitar fuga.

4. Protocolo de entrenamiento y búsqueda

Cada algoritmo se entrenó solo con train, usando StratifiedKFold(n_splits=5, shuffle=True, random_state=42). GridSearchCV maximizó Average Precision. La regresión logística varió regularización C y class_weight; Random Forest varió árboles, profundidad y min_samples_leaf. Los hiperparámetros se determinaron con validación cruzada; los umbrales se eligieron con validación usando F2 y una restricción de precision >= 0.50 cuando resultara factible.

Modelo	*AP de CV*	*F2 validación*	*Umbral*
LogisticRegression	0.4793	0.4167	0.30
RandomForest	0.7347	0.7143	0.20

JUSTIFICACIÓN: con 3.39% de fallas, se optimizó Average Precision en la validación cruzada, en lugar de accuracy; posteriormente se seleccionó el umbral operativo maximizando F2 con precision mínima de 0.50 si existía configuración factible. Esto penaliza especialmente fallas no detectadas sin ocultar el incremento de alertas falsas.

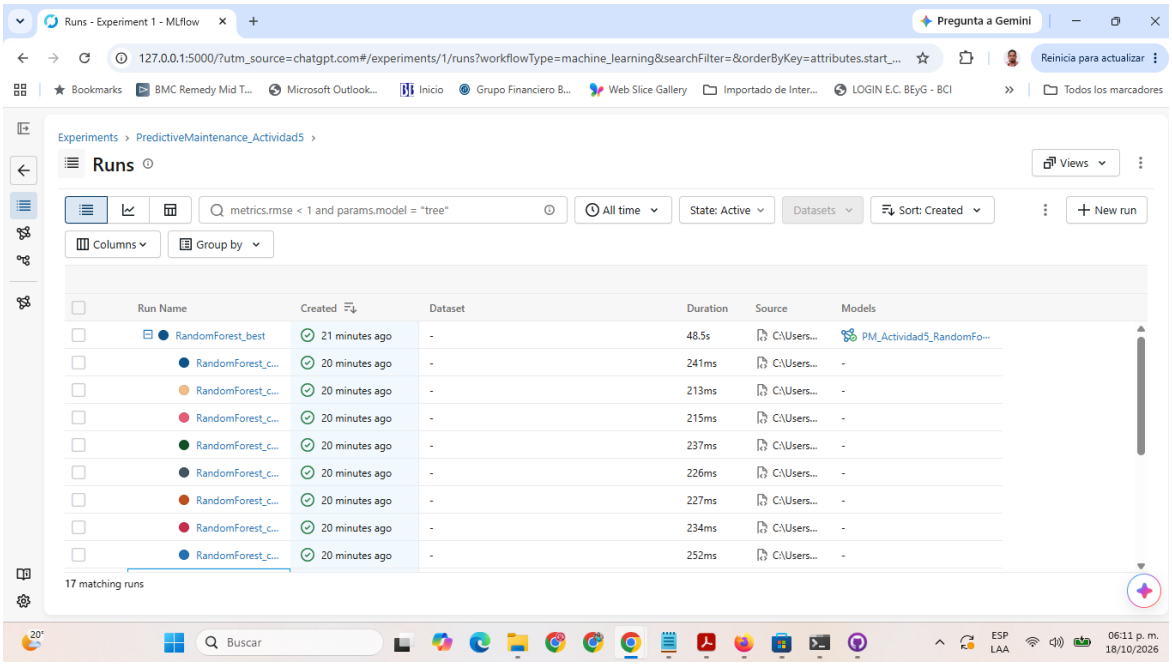
Modelo	*Hiperparámetros elegidos por Grid Search*
LogisticRegression	{'est__C': 1.0, 'est__class_weight': None}
RandomForest	{'est__max_depth': None, 'est__min_samples_leaf': 1, 'est__n_estimators': 240}

La búsqueda evaluó 6 combinaciones de regresión logística (3 valores de C y 2 ponderaciones) y 8 combinaciones de Random Forest (2 árboles, 2 profundidades, 2 tamaños de hoja): 14 candidatos por cinco folds, 70 ajustes de CV. El pipeline reentrenó cada mejor configuración sobre train; la validación externa ajustó el umbral sin utilizar test.

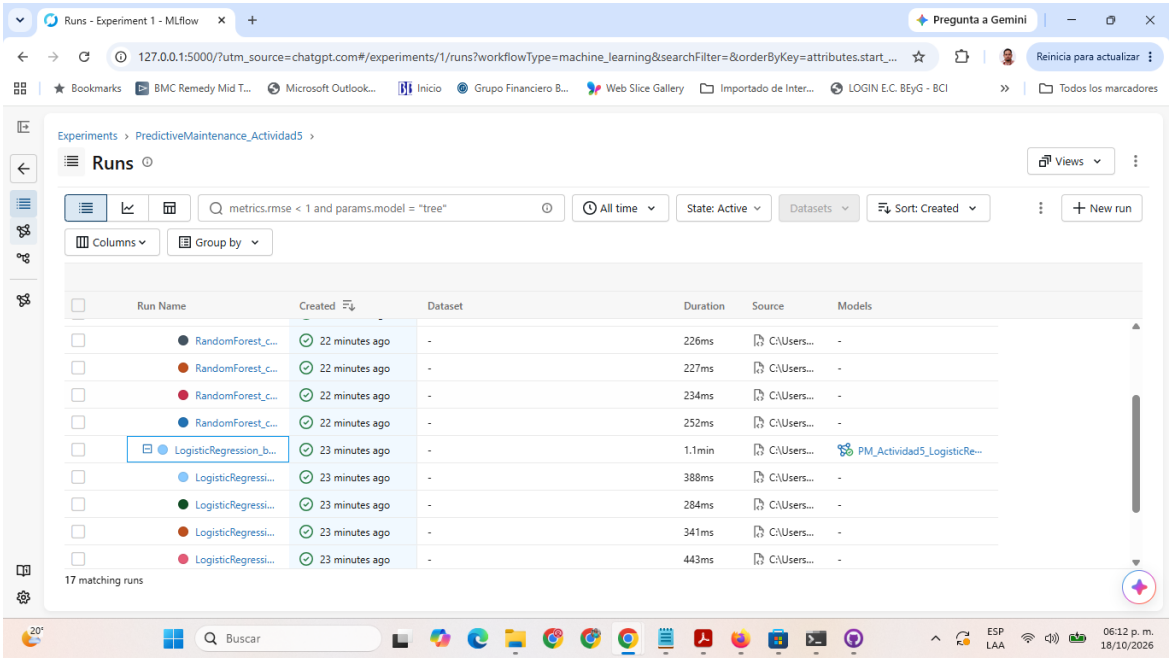
5. Experimentos reproducibles con MLflow

Se registraron 17 runs (14 combinaciones hijas) en el experimento PredictiveMaintenance_Actividad5. Las ejecuciones padre incluyen los mejores modelos completos y sus resultados de validación y prueba; los hijos preservan parámetros y promedios de validación cruzada. Las tablas exportadas son consultables en evidencias/mlflow/.

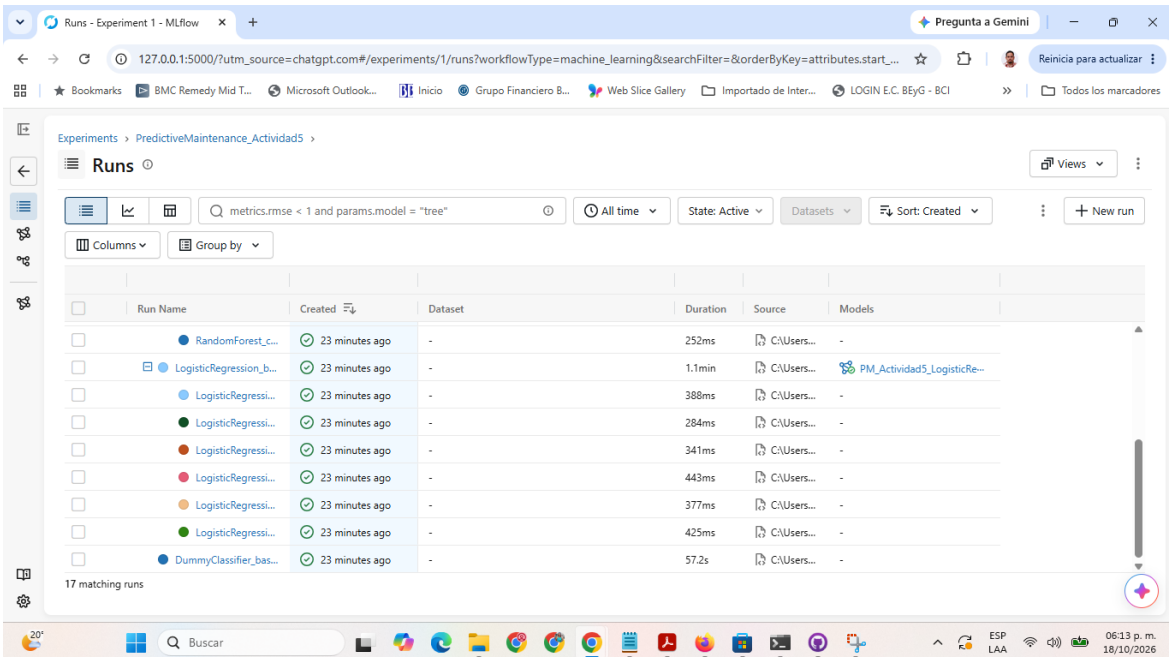
Para la trazabilidad, cada candidato tiene run hijo con parámetros, media/desviación del Average Precision de CV y ranking; cada run padre conserva el pipeline completo, tablas, figuras y el resultado de prueba. El experimento se ejecutó en el servidor MLflow local; los 17 runs también se publican como CSV porque localhost es inaccesible para el evaluador. La evidencia visual se documenta abajo.



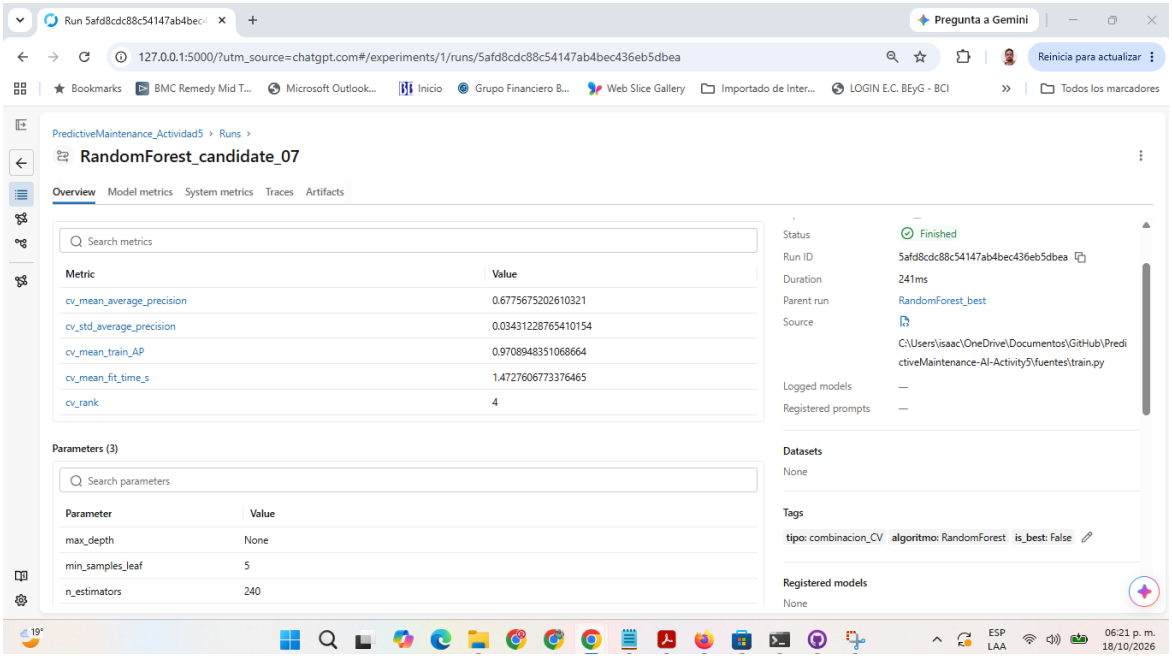
Evidencia MLflow 1a. Listado de runs: parte superior.



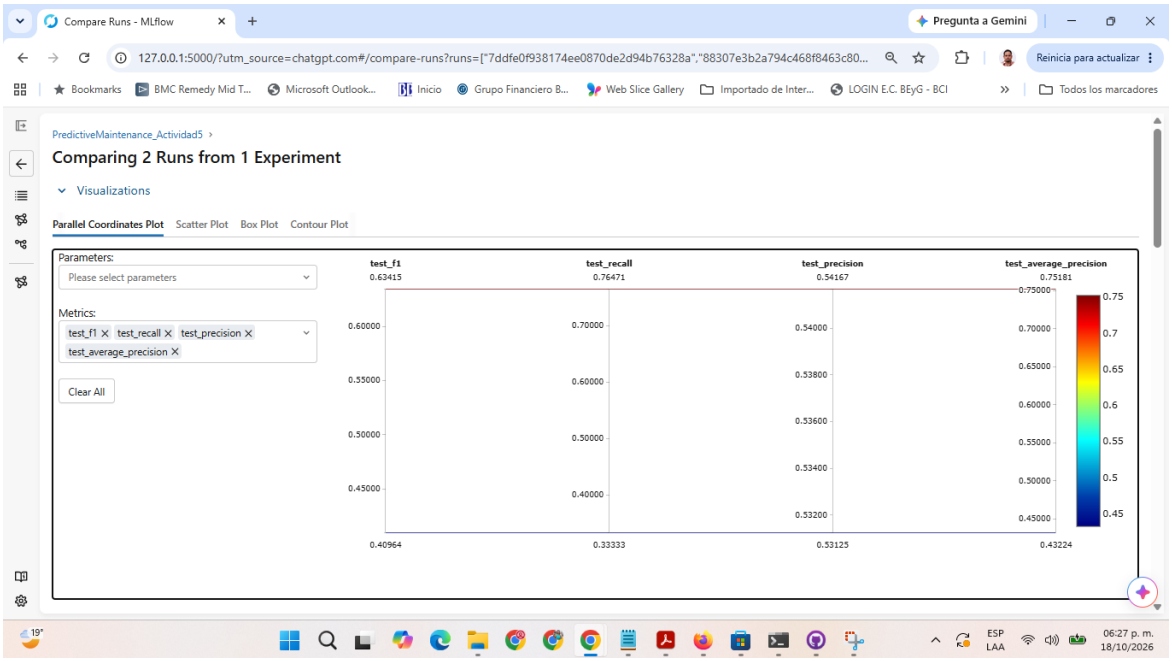
Evidencia MLflow 1b. Listado de runs: continuación.



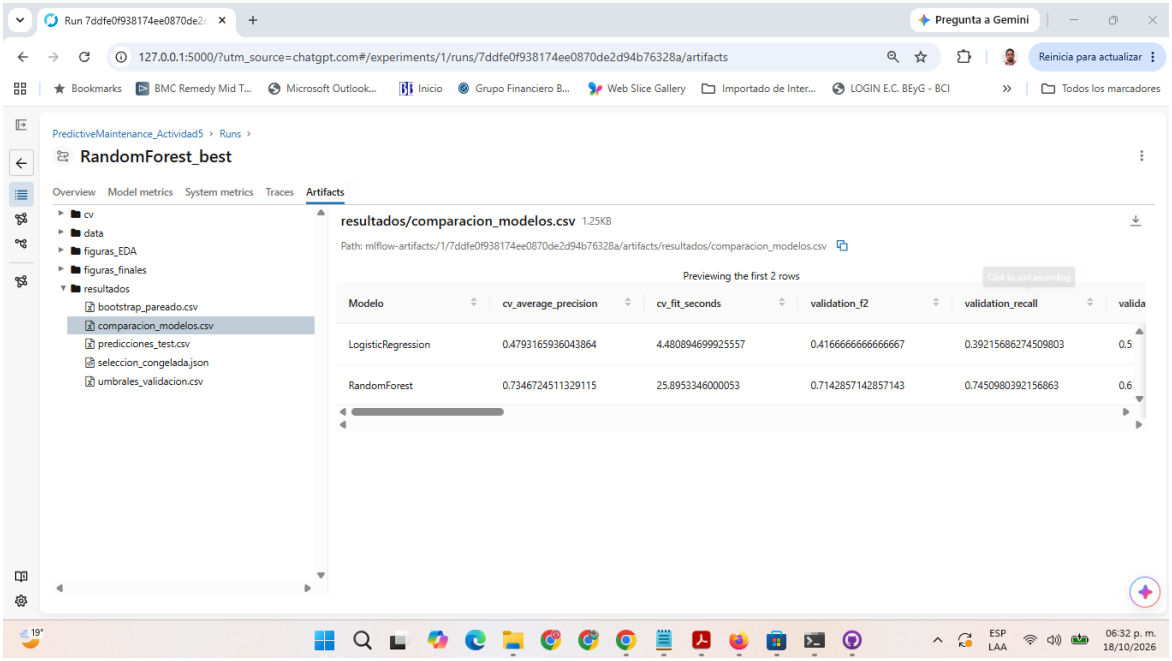
Evidencia MLflow 1c. Listado de runs: tramo inferior.



Evidencia MLflow 2. Hiperparámetros de Grid Search.



Evidencia MLflow 3. Comparación entre modelos.



Evidencia MLflow 4. Modelos registrados y artefactos.

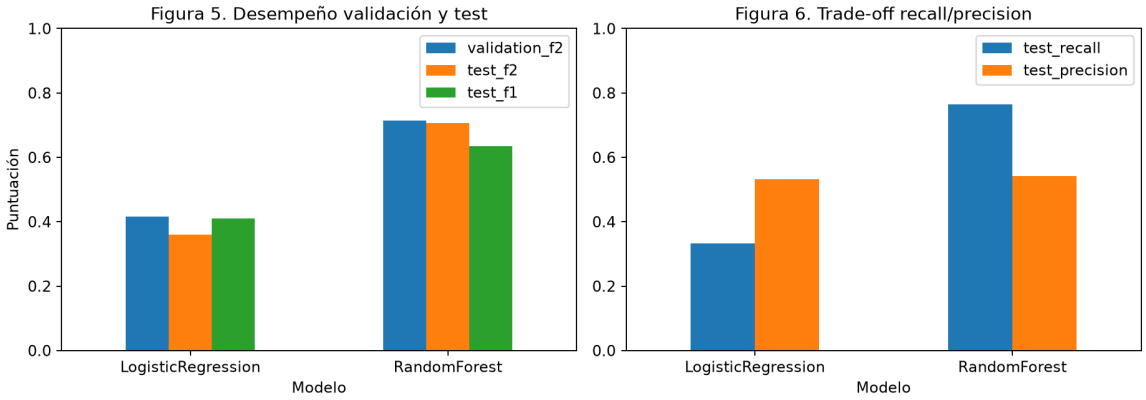
6. Desempeño y contraste de modelos sobre prueba

Modelo	*Precisión*	*Recall*	*F1*	*F2*	*AP*	*ROC-AUC*
LogisticRegression	0.531	0.333	0.410	0.360	0.432	0.901
RandomForest	0.542	0.765	0.634	0.707	0.752	0.960

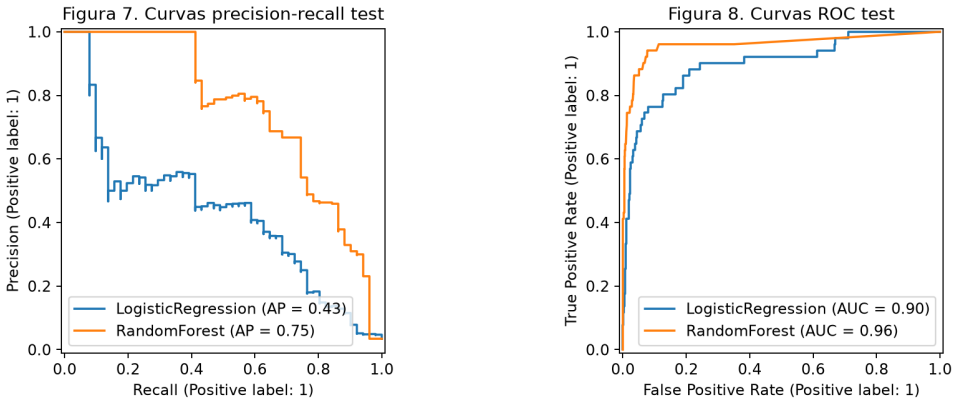
LECTURA CUANTITATIVA DE TEST: Random Forest detectó 39 de 51 fallas; regresión logística detectó 17. Se redujeron los falsos negativos de 34 a 12 (22 fallas adicionales identificadas; 64.7% menos omisiones). En contraparte, las falsas alertas subieron de 15 a 33 (18 inspecciones potenciales adicionales). La precisión fue semejante (54.17% frente a 53.12%), mientras que el recall pasó de 33.33% a 76.47%. En test la mejora absoluta de F1 fue 22.45 puntos porcentuales.

Modelo	*TP*	*FN*	*FP*	*TN*	*Balanced acc.*
LogisticRegression	17	34	15	1434	0.6615
RandomForest	39	12	33	1416	0.8710

BASELINE: DummyClassifier alcanzó 96.6% de accuracy en test prediciendo siempre 'sin falla', pero recall = 0, F1 = 0 y Balanced Accuracy = 0.50. Por tanto, una accuracy superior a 96% no prueba utilidad del clasificador.



Figuras 5 y 6. Comparación de validación y prueba; inspeccionar la posible sensibilidad al umbral.



Figuras 7 y 8. Precision-Recall y ROC, ambas necesarias por el desbalance del target.

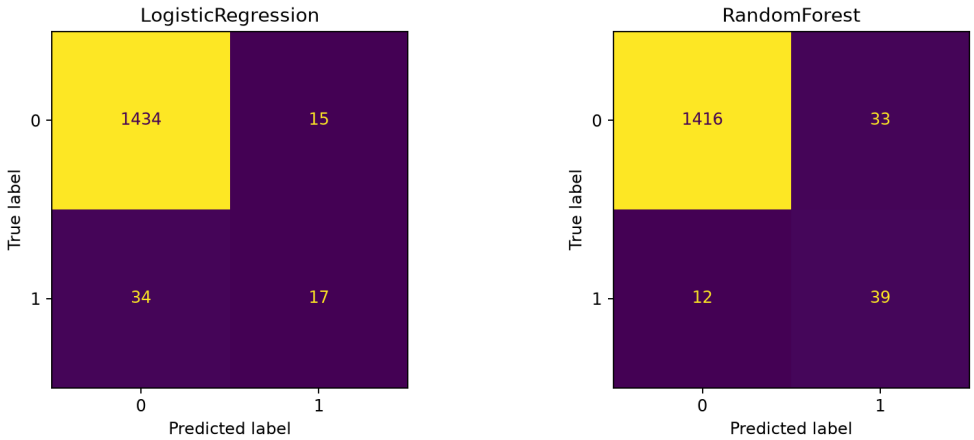


Figura 9. Matrices de confusión para examinar falsos negativos y falsas alarmas.

7. Análisis estadístico e incertidumbre

Bootstrap pareado con 2,000 remuestreos sobre los mismos registros test; delta F1 = RandomForest - LogisticRegression = 0.2245. IC 95% percentil = [0.0897, 0.3668]. El intervalo no incluye cero bajo este procedimiento descriptivo; no constituye validación industrial ni confirmación fuera de la muestra.

Figura 10. Bootstrap pareado de diferencia F1 (2,000)

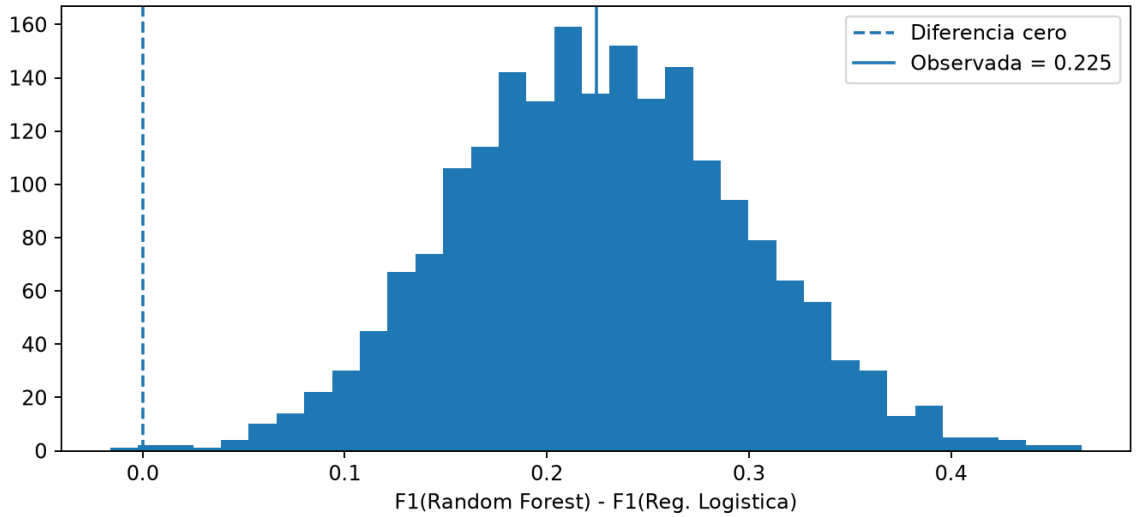


Figura 10. Variación de la diferencia F1 por bootstrap pareado.

INCERTIDUMBRE: la diferencia F1 a favor de Random Forest fue 0.2245 (22.45 puntos); el bootstrap pareado percentil de 2,000 remuestreos produjo IC95 [0.0897, 0.3668]. El intervalo excluye cero bajo este procedimiento, pero los casos positivos son únicamente 51, pertenecen a un dataset sintético y no existe prueba temporal ni industrial; no se infiere superioridad universal.

8. Recursos, deuda técnica y enfoque MLOps

Modelo	*Tiempo GridSearch (s)*	*Latencia por registro (ms)*
LogisticRegression	4.48	0.007
RandomForest	25.90	0.037

Deuda técnica: snapshots académicos sin datos temporales de equipos, ausencia de monitoreo de distribución (drift), degradación por máquina, validación prospectiva y cuantificación real de costo por falso negativo. Acciones MLOps: versionar datos/pipelines, registrar nuevos datasets y modelos en MLflow, definir reglas de promoción, monitorear métricas por segmento, revisar drift, auditar alertas y mantener reversión humana.

LECTURA DE EFICIENCIA Y GENERALIZACIÓN: Random Forest necesitó 25.90 s para Grid Search frente a 4.48 s de regresión logística en el hardware observado (~5.8 veces más), y 0.0366 frente a 0.0067 ms por registro de prueba. La menor latencia de regresión logística no compensó la omisión de 34 fallas. La configuración ganadora de Random Forest obtuvo AP de entrenamiento cercano a 1.0, frente a 0.7347 de CV: señal de posible sobreajuste que exige validación externa antes de un despliegue.

SENSIBILIDAD ECONÓMICA ILUSTRATIVA (sin equivalencia en pesos): si una falsa alarma cuesta 1 unidad y una falla omitida 10 unidades, el costo observado sería RF = 33 + 10x12 = 153 y logística = 15 + 10x34 = 355. Los costos de una planta real se desconocen; estos números no justifican por sí solos invertir.

PROPUESTA MLOps POR ETAPAS: (1) publicar manifests SHA256 y modelo por versión, (2) automatizar pruebas y validación de schema, (3) monitorear drift de velocidad, torque y prevalencia de fallas, (4) medir recall/precision por máquina y periodo, (5) definir rollback y aprobación humana antes de cualquier acción sobre maquinaria.

9. Conclusión crítica

Con el protocolo preestablecido se seleccionó RandomForest sobre validación, antes de evaluar test. El modelo seleccionado debe entenderse como una línea base académica sobre datos sintéticos, no un sistema listo para anticipar fallas en una planta. Los resultados, sus intervalos, y los runs MLflow permiten evaluar reproducibilidad y sostener decisiones sin exagerar su generalización.

10. Declaración de uso de inteligencia artificial

Herramienta	*Uso declarado*
ChatGPT (OpenAI)	Diseño experimental, asistencia en scripts Python, documentación, depuración y control de versiones; las métricas proceden de ejecuciones reales verificables.
Responsabilidad humana	Configuración, ejecución y revisión de resultados, interpretación académica y publicación final.

11. Evidencias, reproducibilidad y límites

Repositorio: datos/datos_ini, datos/datos_limp, fuentes/train.py, fuentes/entrena.ipynb, resultados/, evidencias/mlflow/ y docs/Reporte_Tecnico_Actividad5.pdf. Para reproducir: instalar requirements.txt, iniciar MLflow en 127.0.0.1:5000, ejecutar python -m fuentes.train y después python -m fuentes.export_evidencias. Los umbrales quedan congelados antes de calcular prueba.

DOCUMENTACIÓN COMPLEMENTARIA: fuentes/Evidencia_Ejecutada_Actividad5.ipynb contiene verificaciones ejecutadas sin volver a entrenar; fuentes/entrena.ipynb y fuentes/train.py contienen el procedimiento reproducible de entrenamiento. El CSV original UCI y el manifiesto de correspondencia completan la procedencia del dataset.

La actividad cubre un flujo académico de MLOps; NO se valida tiempo hasta fallo, sensores reales, desgaste por activo, causalidad, deriva futura, eficiencia bajo carga productiva ni costos reales de mantenimiento. No se almacenan tokens ni el SQLite privado en GitHub.

12. Bibliografía y recursos

UCI AI4I 2020, DOI https://doi.org/10.24432/C5HS5C. Scikit-learn: https://scikit-learn.org/. MLflow Tracking: https://mlflow.org/docs/latest/ml/tracking/. Repositorio y código: https://github.com/isaaccg88/PredictiveMaintenance-AI-Activity5